

## Module #1: Spectrum Spider

### Function **\_\_init\_\_**(self, max\_files=100, \*args, \*\*kwargs)

**\*\*kwargs**: prevents scrapy.Spider crashes by collecting keyword arguments ¶ function signature.

**\*args**: prevents scrapy.Spider crashes by collecting positional arguments ¶ function signature.

**max\_files**: the maximum number of PDF document files to download and process (default 100).

**self**: the SpectrumSpider instance that invokes the method.

**self.docs\_metadata**: a list of dictionaries storing metadata (title, degree, year, etc) for each PDF.

**self.docs\_tokens**: a list of term lists associated with each document, convenient for later TF-IDF.

**self.files\_downloaded**: a counter tracking how many PDFs have been processed thus far.

**self.index**: a dictionary representing the inverted index that is incrementally constructed.

**self.max\_files**: the maximum number of PDF document files to download and process (default 100).

**self.output\_dir**: directory path "output" for storing all post-crawling results.

**self.pdf\_dir**: directory path "pdfs\_extracted" for temporarily storing downloaded PDFs.

Purpose: initializes the spider with configurable file limits and sets up the directory structure, the inverted index, and the data structures for collecting document metadata and tokens during the crawling pipeline. Basically the SpectrumSpider constructor.

### Function **start\_requests**(self)

**self**: the SpectrumSpider instance that invokes the method.

**callback**: the function that receives and processes the HTTP response from the homepage.

Purpose:

Returns a scrapy.Request object containing the URL to fetch and the callback function.

Initiates the crawling pipeline by issuing the first HTTP request to the Spectrum homepage.

The retrieved HTML is passed onto the parse\_homepage() to continue the pipeline.

### Function **parse\_homepage**(self, response)

**self**: the SpectrumSpider instance invoking the method.

**doctype\_url**: the URL extracted from the homepage's "by Document Type" link

**response**: the HTTP response object from the Spectrum homepage.

**callback**: the function that receives and processes the HTTP response from the homepage.

Purpose:

Locates the "by Document Type" link found under the homepage's "Browse" section.

Issues a follow-up request for the extracted URL so that the crawling pipeline can continue.

Delegates handling of the next response to the function parse\_document\_types().

Includes exception handling in the event the link could not be found, which stops the crawl.

### Function **parse\_document\_types**(self, response)

**self**: the SpectrumSpider instance invoking the method.

**thesis\_url**: the URL extracted from the "Thesis" entry under the "Browse by Document Type" list.

**response**: the HTTP response object for the "By Document Type" page.

**callback**: the function that will receive and process the follow-up HTTP request.

#### Purpose:

Searches through the "Browse by Document Type" list and locates the "Thesis" link.

Issues a follow-up request for the extracted URL so that the crawling pipeline can continue.

Delegates handling of the next response to the function `parse_document_types()`.

Includes exception handling in the event the link could not be found, which stops the crawl.

### Function **parse\_thesis**(self, response)

**self**: the SpectrumSpider instance invoking the method.

**degree\_label**: the current degree category being searched for (being "Masters" or "Phd").

**degree\_url**: the URL extracted from the link corresponding to the given degree category.

**response**: the HTTP response object for the "Thesis" page.

**callback**: the function that will receive and process the follow-up HTTP request.

**cb\_kwargs**: a dictionary carrying metadata forward to the next stage of the crawl.

#### Purpose:

Identifies the subdirectories under the Thesis category, being "Masters" and "Phd" respectively.

For each `degree_label`, the function extracts the corresponding `degree_url` assuming it exists.

Also passes metadata forward so that later functions of the pipeline know which branch was taken.

Issues a follow-up request for the extracted URL so that the crawling pipeline can continue.

Delegates handling of the next response to the function `parse_thesis_years()`.

### Function **parse\_thesis\_years**(self, response, degree)

**self**: the SpectrumSpider instance that invokes the method.

**response**: the HTTP response object for the Masters of PhD thesis listing page.

**degree**: a string label metadata indicating the degree type for the current branch of the crawl.

**year\_links**: the list of URLs extracted from links whose text is a four-digit year between 1900-2030.

**callback**: the function that receives and processes the HTTP response from the homepage.

**cb\_kwargs**: a dictionary carrying metadata forward to the next stage of the crawl.

#### Purpose:

Scans the current degree page for links whose visible text is a valid year between 1900-2030.

Collects all these links under a list `year_links`, then iterates over each year contained in this list.

For every valid link found, a follow-up request is issued for that page so the pipeline can continue.

Also passes metadata forward so that later functions of the pipeline know which branch was taken.

Delegates handling of the next response to the function `parse_article_list()`.

### Function **parse\_article\_list**(self, response, degree, year)

**self**: the SpectrumSpider instance that invokes the method.

**response**: the HTTP response object for the specific thesis-year page from the previous function.

**degree**: a string label metadata indicating the degree type for the current branch of the crawl.

**year**: a 4-digit string metadata indicating the publication year for the current branch of the crawl.

**article\_links**: the list of URLs extracted from links pointing to individual thesis article pages.

**callback**: the function that receives and processes the HTTP response from the homepage.

**cb\_kwargs**: a dictionary carrying metadata forward to the next stage of the crawl.

#### Purpose:

Scans the current year page for individual article links in order to download their PDFs next.

Collects all these links under a list `article_links`, then iterates over each article contained in this list.

For every valid link found, a follow-up request is issued for that page so the pipeline can continue.

Also passes metadata forward so that later functions of the pipeline know which branch was taken.

Delegates handling of the next response to the function `parse_article_list()`.

Stops crawling immediately if the maximum number of PDF files is reached.

### Function **parse\_article**(self, response, degree, year)

**degree**: a string indicating the degree type (Master or PhD) passed through the callback chain.

**pdf\_complete\_link**: the string absolute URL that points to the PDF download link.

**pdf\_link**: the string relative URL that points to the PDF download link.

**response**: a Scrapy response object containing the HTML of an individual thesis article page.

**self**: the SpectrumSpider instance that invokes the method.

**title**: the human-readable title of the thesis pulled from the article page.

**year**: a 4-digit string indicating the publication year passed through the callback chain.

Purpose: extracts the thesis title and PDF download link from the given article page, then issues a request to download the PDF all while passing along all metadata. Ensures the file limit has not been exceeded before executing.

### Function **extract\_pdf\_text**(self, pdf\_path)

**page**: the individual PDF document page object being processed in the loop

**pdf\_path**: a string filepath to a PDF document file presumably stored in the `pdfs_extracted` folder.

**self**: the SpectrumSpider instance that invokes the method.

**text**: a string that accumulates all raw text extracted from the PDF document.

Purpose: opens a PDF document file using the PyMuPDF (fitz) library, iterates through each page to extract text content, and returns the accumulated text for future processing including term extraction, inverted index construction, and clustering analysis.

**Function** `parse_pdf(self, response, degree, year, title, pdf_url, spectrum_article_url)`

**doc\_id**: a unique integer identifier assigned to the given PDF document file given its metadata.

**degree**: a string indicating the degree type (Master or PhD) passed through the callback chain.

**pdf\_url**: a string relative URL from which the PDF was downloaded.

**response**: a Scrapy response object containing the binary PDF file data plus associated metadata.

**self**: the SpectrumSpider instance that invokes the method.

**spectrum\_article\_url**: a string absolute URL from which the PDF was downloaded.

**temp\_pdf\_path**: a temporary string file path where the PDF is saved during processing.

**terms**: the list of preprocessed terms returned by `update_index()` from module #3.

**text**: the extracted raw text content from the given PDF document file.

**title**: a string containing the URL thesis title passed through the callback chain.

**year**: a 4-digit string indicating the publication year passed through the callback chain.

**Purpose**: downloads and temporarily saves the PDF document file, extracts its text, updates the inverted index with the document's terms, stores document metadata and tokens for later clustering, then deletes the temporary file. It is also at this stage when the upper bound limit is incremented and the pipeline verifies whether it has been exceeded or not. If it has, the crawling process terminates.

**Function** `close(self)`

**f**: file handle for writing to a JSON file.

**index\_path**: a path object that defines where the inverted index JSON file will be saved.

**meta\_path**: a path object that defines where the document metadata JSON file will be saved.

**self**: the SpectrumSpider instance that invokes the method.

**tokens\_path**: a path object that defines where the document tokens JSON file will be saved.

**Purpose**: executes when the crawling pipeline terminates, saving the inverted index, document metadata, and per-document tokens to distinct JSON files in the output directory for later use in querying and clustering. The JSON files are also highly useful for debugging.

## Design Decisions

The Scrapy library is responsible for automatically crawling the Spectrum library's HTML structure and downloading the thesis article PDFs. It was chosen over the other libraries because it handles the standard for robot exclusion criteria conveniently, is faster than most other tools, automates a good portion of the crawling pipeline implementation, and is widely used in professional contexts. In order to ensure that results were deterministic, I purposefully set `CONCURRENT_REQUESTS=1`.

The PyMuPDF library is responsible for opening each temporarily downloaded PDF and extracting their raw textual content in order to construct the inverted index. Note that "fitz" is the Python name for the library.

A large number of PDFs from the Spectrum repository will fail to extract because the thesis articles are really scanned images that the PyMuPDF library simply cannot parse for textual content. Given the feedback from the discussion board and the difficulty of extracting text from images in Python, these articles are ignored by the crawler despite seemingly being the majority.

## Module #2: Crawler Indexer

### Function `load_inverted_index()`

Purpose: a modified version of the `load_json_index()` function from module #4 that raises an error if the "inverted\_index.json" file does not exist, which normally happens because the web crawler has not been executed first.

### Function `modified_lookup_singleQ(index, term)`

**index**: the (term, docID) dictionary representing the inverted index to search through.

**term**: the query string provided by the user that needs to be normalized.

**docIDs**: the sorted deduplicated list of document IDs that contain the normalized query term.

**elapsed**: the total number of seconds that was needed to process the query.

Purpose: a modified version of the `lookup_singleQ()` function from module #4 that returns a sorted and deduplicated list of unique document IDs, as well as ignores the performance analysis statistic for simplicity.

### Function `build_mycollection(index)`

**index**: represents the inverted index dictionary mapping terms to their postings lists.

**sustainability\_docIDs**: set of document IDs containing the term "sustainability" returned by query.

**waste\_docIDs**: set of document IDs containing the term "waste" returned by query.

**intersection\_docIDs**: the sorted list of document IDs containing both "waste" and "sustainability".

**My\_collection**: the sorted list of document IDs containing either or both "waste" and "sustainability".

Purpose: builds the instructed document collection "My\_collection" by querying for "sustainability" and "waste" individually then merging the two document collections using set union. Also prints how many documents were returned for each query, how many documents were returned that have both query terms, and how many documents (excluding duplicates) were returned in total.

**Function** `build_tfidf_matrices`(document\_tokens, Mycollection\_docIDs)

**document\_tokens**: a list of strings representing a sequence of space-joined tokens from one PDF.

**Mycollection\_docIDs**: a list of integer document IDs returned by `build_mycollection()`.

**Mycollection\_tokens**: a list of space-joined token sequences for only the documents in Mycollection.

**term\_names**: array of all vocabulary terms extracted during global training, TF-IDF vector columns.

**vocab**: dictionary mapping each vocabulary term to its column index in the TF-IDF matrix.

**vectorizer\_global**: A `TfidfVectorizer` trained on all crawled documents and consequently its IDF values are learned over all crawled documents.

**vectorizer\_local**: A `TfidfVectorizer` trained on only those documents belonging to `My_collection` and so its IDF values are learned only over this subset rather than the full corpus.

**X\_global**: TF-IDF matrix of only those documents belonging to `My_collection`, but using IDF values learned from the entire crawled corpus. (`#My_collection docs × #vocab terms`).

**X\_local**: TF-IDF matrix of only those documents belonging to `My_collection`, but using IDF values learned only from the `My_collection` set of documents.

**Purpose**: constructs two TF-IDF representations of the same subset of documents so their results can be later clustered and compared, following the instructions of the bonus point. The matrix(`ces`) returned by this function are necessary to run `execute_kmeans()`.

**Function** `execute_kmeans`(X, term\_names, k\_values, writer)

**centers**: a 2D array ( $K \times \text{\#terms}$ ) representing the cluster centroids in TF-IDF vector space.

Each row represents the centroid vector for cluster `c`.

Each column corresponds to a specific term belonging to the `term_names` parameter.

Value `centers[c, j]` represents the average TF-IDF weight of term `j` across all documents in cluster `c`.

**cluster\_index**: the unique integer ID of the current cluster being processed in the second loop.

**cluster\_size**: the number of documents assigned to the current cluster.

**k\_values**: a sequence of `k` values (representing the number of clusters) to test typically [2,10,20].

**k**: the current number of clusters being tested in the loop.

**km**: a `KMeans` clustering model object configured with the current `k` value from the sequence.

**labels**: a numpy array where each entry is the cluster ID assigned to the corresponding document.

**order\_centroids**: a 2D array with term columns sorted by descending weight for each cluster.

**term\_index**: the unique integer ID of the current term being processed in the third loop.

**term\_names**: a numpy array of vocabulary terms corresponding to matrix columns.

**top\_indices**: a numpy array of length 50 for the top 50 terms (in weight) for the given cluster.

**writer**: an optional file handle for writing results, if `None` prints to console by default.

**X**: the TF-IDF matrix where rows are documents and columns are term weights.

**Purpose**: trains and executes a K-means clustering model for each specified value of `k`, using the TF-IDF matrix returned by `build_tfidf_matrices()`. For every cluster, it ranks all vocabulary terms by their TF-IDF weight within the centroid and prints the top 50 from among them. Also optionally allows results to be written to an external file for more convenient viewing, especially when later interpreting cluster themes.

## Clustering Analysis

When  $k = 2$ , the algorithm separates the collection into two clusters that seem to align closely with the two major academic disciplines offered by Concordia University: "Social sciences & Humanities" and "Engineering & Science". This grouping is reasonable because the query terms "sustainability" and "waste" can be interpreted differently depending on the discipline. In the social sciences, they tend to appear in contexts discussing public policy and human behavior. In engineering, they appear in contexts discussing material & energy efficiency and system optimization.

When  $k = 10$ , the algorithm yields more focused and internally consistent clusters. Although the core distinction between disciplines lingers, highly specialized and coherent research themes or subjects are now clearly discernable in the results. Here are some noteworthy examples:

Cluster #0 delves into cellular neuroscience (dopamine, receptor, striatal, amphetamine, locomotor).

Cluster #3 delves into media studies (artist, aesthet, creativ, polit, cultur, discours, film).

Cluster #5 delves into mechanical engineering and fluid dynamics (deflect, damp, stiff, fluid, veloc).

Cluster #6 clearly groups together french-language articles (qui, nous, aussi, une, mouvement).

Cluster #7 delves into corporate finance (firm, ceo, equiti, debt, investor, cash).

Cluster #8 delves into sustainable energy systems (thermal, solar, environment, water, sustain).

When  $k = 20$ , the specialization trend continues.

By inspection, many of the broader thematic or topical clusters from the  $k = 10$  run have been further fragmented into narrower sub-topics. However, the size of each cluster is noticeably less consistent, with some containing just one or two documents and some containing six or eight. This imbalanced distribution might suggest that the algorithm is simply trying too hard to identify meaningful clusters from a collection that just doesn't have that many. In other words, it has been over-partitioned, and better results are likely generated by a value of  $k$  closer to 10.

Some noteworthy examples:

Cluster #01 delves into sustainable thermal-energy systems (thermal, exergi, hvac, refriger, ventil).

Cluster #05 delves into corporate finance and social responsibility (firm, ceo, equiti, csr, gdp, debt).

Cluster #07 delves into attention and brain research (lobe, frontal, cognit, hyponsi, brain, cortex).

Cluster #09 delves into computer vision and robotics (camera, pixel, frame, sensor, matlab, angl).

Cluster #10 clearly groups together french-language articles (qui, nou, aussi, une, mouvement).

Cluster #11 delves into everyday social and cognitive psychology (social, belief, feel, preceiv, listen).

Cluster #14 delves into freshwater and marine ecology (aquat, salmon, lake, sediment, river).

Cluster #16 delves into film studies and cultural critique (artist, asthet, film, cinema, hollywood).

As a general overview, most clusters are coherent and cohesive with subjects that are easy enough to understand once certain keywords are identified. However, without having necessarily read each of the crawled articles, it isn't immediately clear how each cluster is related to the search terms. It's possible that "waste" and/or "sustainability" do appear in the document but that they aren't the core idea being discussed, leading to potentially irrelevant groupings for the user's information need. It also emphasizes that some terms that seem straightforward at first glance have different meanings or connotations according to the discipline. Assuming that the project instructions intended to return documents relating to environmental sustainability and waste, the findings suggest that it would be better to explicitly include the term in the search query.

When comparing the global and local TF-IDF results, the key difference really comes from how each approach measures term importance. The global TF-IDF is trained on every crawled PDF thesis, so its inverted document frequency (IDF) values represent how rare a term is across the entire corpus. The practical result is that uncommon and oftentimes domain-specific vocabulary stand out, hence clusters end up cleanly separated into the major academic fields offered at Concordia. Conversely, the local TF-IDF is trained only on documents belonging to My\_collection, a much narrower subset that already shares many waste or sustainability related topics and of course vocabulary. Since they have so much thematic overlap, the term weighting is spread more evenly and fewer terms will stand out. In practice, the local clusters still produce similar themes as their global counterparts, but their boundaries are noticeably softer and their top terms more generic than strongly tied to a specific discipline.

For example, take the top 10 (stemmed) terms from local cluster #14 when  $K = 20$ :

- fish
- fisheri
- aquat
- ecolog
- river
- boat
- stock
- firm
- marin
- island

These are all very generic within the (presumed) context of environmental or ecological research focused on sustainability efforts. Sure they might be specialized enough that a reader can guess that the environment in question is aquatic, but it's impossible to tell the specific topic(s) examined by the six documents in the cluster. Compared to its counterpart global cluster #14 when  $K = 20$ :

- aquat
- salmon
- lake
- sediment
- river
- anthropogen
- trout
- freshwater
- fish
- spm

These terms are observably less generic, with those like "salmon", "trout", "freshwater", "sediment", and "anthropogen" pointing towards specific thesis topics and contexts. They don't just indicate that the documents discuss the broad subject of aquatic environments like the local cluster does, instead they narrow the focus to recognizable subfields within the broader academic theme.



## Module #3: Crawler Indexer

**TOKENIZER:** a regex tokenizer that keeps alphanumeric characters and apostrophes.

**STOPWORDS:** the NLTK English stopwords set, used to filter common but uninformative words.

**STEMMER:** a Porter stemmer instance used to reduce words to their root by removing suffixes.

### Function preprocess\_tokenize(text)

**stemmed\_term:** the root or stem of an individual token after applying the STEMMER.

**terms:** the final list of preprocessed terms following the filtering and stemming stages.

**text:** the raw document text string that needs to be preprocessed, extracted from the PDF.

**tokens:** the list of raw tokens extracted from the raw document text using the TOKENIZER.

Purpose: transforms raw document text into a list of preprocessed terms ready for indexing by applying linguistic normalization techniques like case folding, tokenization, stopwords elimination, number elimination, and stemming.

### Function update\_index(inverted\_index, doc\_id, text)

**doc\_id:** unique integer identifier for the current document (PDF) being processed.

**inverted\_index:** dictionary mapping each term to a list of document IDs where it appears.

**postings:** the postings list (list of document IDs) associated with the specific term.

**seen:** a set tracking terms already encountered in the document to prevent duplicate docIDs.

**term:** the individual term being processed in the loop.

**terms:** list of preprocessed terms extracted from the raw text via preprocess\_tokenize().

**text:** the raw text content extracted from the document (PDF).

Purpose: incrementally builds the inverted index throughout the crawling process by adding the current document's ID to the postings list of each unique term it contains, while also maintaining per-document term lists for later clustering analysis. Regularly called in parse\_pdf() from module #1.

### Function build\_inverted\_index(pairs)

**docID:** the unique integer document identifier from each (term, docID) pair.

**index:** the inverted index dictionary mapping terms to postings to be constructed by the function.

**pairs:** the list of (term, docID) pair tuples to be organized into an inverted index.

**term:** the string term from each (term, docID) pair.

Purpose: constructs a complete naive inverted index from a collection of term-document pairs. It is never invoked at any point throughout the project, and is here just to flesh out the class.

## Design Decisions

The architecture of the inverted index was essentially ported over from its naive implementation in the first project, excluding the SGM parsing logic because it is irrelevant to this project. Furthermore, the project instructions suggested that the inverted index be built incrementally during the execution of the crawling pipeline and then saved to a JSON file. Both recommendations were followed in this implementation.

## Module #4: Query Processor

**STEMMER:** a Porter stemmer instance used to reduce words to their root by removing suffixes.

**Function** `_normalize`(term: str) → str

**term:** the raw input query string provided by the user that needs to be normalized.

**Purpose:** standardizes query terms to match the preprocessing stages applied during index construction, being case-folding and stemming. Ensures consistent retrieval results.

**Function** `intersect_postings`(p1, p2)

**answer:** a list that accumulates document IDs that appear in both input postings lists.

**i:** an index pointer tracking the current position in the first postings list p1.

**j:** an index pointer tracking the current position in the second postings list p2.

**p1:** the first postings list containing document IDs in sorted order.

**p2:** the second postings list containing document IDs in sorted order.

**Purpose:** implements the intersection algorithm from textbook figure 1.6 which identifies those documents that are common to both postings lists, essential for processing AND queries later on with function `lookup_andQ` ( ).

**Function** `lookup_singleQ`(index: Dict[str, List[int]], term: str) → List[int]

**elapsed\_time:** the total number of seconds that was needed to process the query.

**end\_time:** timestamp when query processing concludes to calculate total processing time.

**index:** the (term, docID) dictionary representing the inverted index to search through.

**start\_time:** timestamp when query processing begins to calculate total processing time.

**result:** the sorted list of document IDs that contain the normalized query term.

**term:** the query string provided by the user that needs to be normalized.

**Purpose:** processes single-term queries by retrieving all documents containing the specified term from the given inverted index. Also returns performance analysis statistics.

**Function** `lookup_andQ`(index: Dict[str, List[int]], \*term: str) → List[int]

**\*terms:** a tuple of query strings provided by the user that need to be normalized.

**elapsed\_time:** the total number of seconds that was needed to process the query.

**end\_time:** timestamp when query processing concludes to calculate total processing time.

**index:** the (term, docID) dictionary representing the inverted index to search through.

**intersect\_result:** the working set of document IDs that satisfies all terms processed thus far, which progressively shrinks as each additional term's postings list is intersected with it.

**postings\_list:** the sorted postings list retrieved from the index for a particular term.

**start\_time:** timestamp when query processing begins to calculate total processing time.

**term\_postings:** a list that stores the postings lists for all query terms in \*terms.

**Purpose:** processes multiple term queries by retrieving all documents containing the specified terms from the given inverted index. Also returns performance analysis statistics. Implements the conjunctive query intersection algorithm from textbook fig 1.7.

### Function load\_json\_index(path)

**data:** the raw dictionary loaded in from the JSON file (terms mapped to postings lists).

**f:** file handle for reading the JSON file.

**path:** a string filepath to the JSON file containing the serialized inverted index built in module #1.

**postings:** the list of document IDs associated with a given term, converted from JSON to integers.

**term:** the individual term key belonging to the loaded dictionary.

Purpose: reads the inverted index stored in the JSON file and reconstructs it into a dictionary where each term maps to a postings list of integer document IDs. Important for resolving type mismatches and ensuring full compatibility with functions in module #1 and #2.

### Design Decisions

Similar to module #3, the architecture of the query processor was essentially ported over from its implementation in the first project, with only minimal changes made beyond including JSON loading and parsing logic. Its supporting functions are important components of the clustering pipeline that is implemented in module #2.